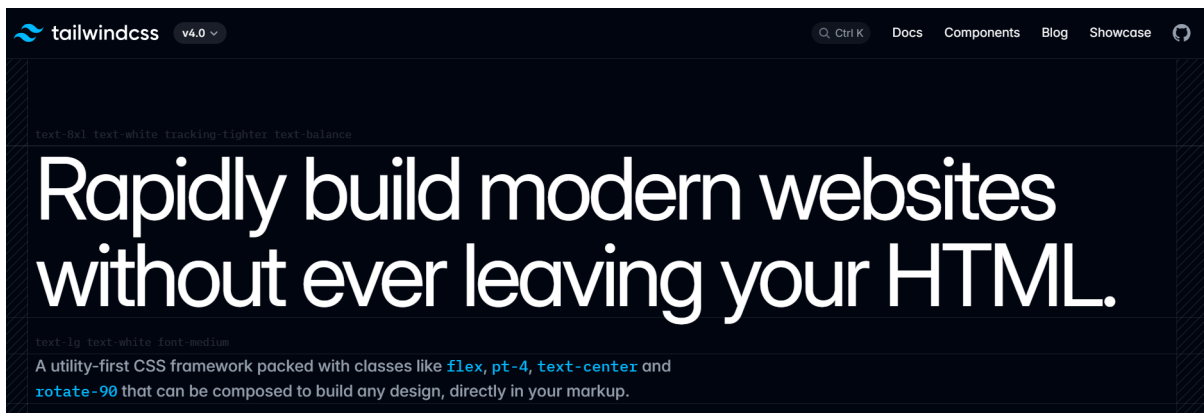




Pertemuan 3

▼ Perkenalan Tailwind CSS



<https://tailwindcss.com/docs>

Tailwind CSS merupakan framework *utility-first* yang memungkinkan kita membangun UI dengan cepat menggunakan kelas CSS yang sudah disediakan, tanpa perlu membuat class CSS manual.

CSS Manual

```
.div {  
  background-color: red;  
  padding-right: 5px;  
}
```

Tailwind CSS

```
<div class="bg-red-50 pr-4">
```

Class yang sudah disediakan oleh tailwindcss sudah sangat lengkap sehingga kita bisa membuat styling yang sederhana maupun styling mengikuti desain yang kita miliki.

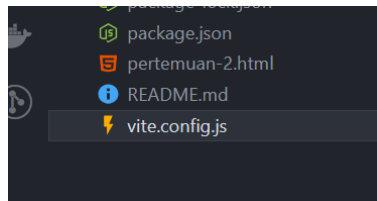
▼ Install Tailwind CSS 4

Untuk menginstall tailwind css, silahkan ikuti langkah-langkah berikut

- install package yang dibutuhkan:

```
npm install tailwindcss @tailwindcss/vite
```

- buka file `vite.config.js` yang ada di direktori utama lalu tambahkan baris yang di-highlight berikut:



```
vite.config.js > ...
1  import { defineConfig } from 'vite'
2  import react from '@vitejs/plugin-react'
3  import tailwindcss from '@tailwindcss/vite'
4
5  // https://vite.dev/config/
6  export default defineConfig({
7    plugins: [
8      react(),
9      tailwindcss(),
10   ]
11 })
12
```

- Buatlah folder `pertemuan-3` lalu buat didalamnya beberapa file berikut:

`main.jsx`

```
1  import { createRoot } from "react-dom/client";
2  import TailwindCSS from "../TailwindCSS";
3  import './tailwind.css';
4
5  createRoot(document.getElementById("root"))
6    .render(
7      <div>
8        <TailwindCSS/>
9      </div>
10 )
11
```

`tailwind.css`

```
src > pertemuan-3 > tailwind.css
1  @import "tailwindcss";
```

Isi file custom.css hanya 1 baris ini saja...

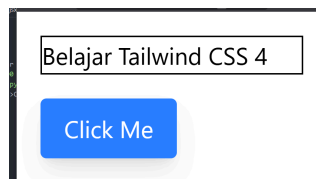
Pastikan import

`./tailwind.css`

`TailwindCSS.jsx`

```
1  export default function TailwindCSS() {
2    return (
3      <div>
4        <h1 class="border m-4">Belajar Tailwind CSS 4</h1>
5        <button className="bg-blue-500 text-white
6          px-4 py-2 mx-4 rounded
7          shadow-lg">Click Me</button>
8      </div>
9    )
10 }
11
```

- Hasil penerapan class tailwind pada sebuah Button beserta penjelasannya



<code>border</code>	Memberikan garis tepi (border) default.
<code>m-4</code>	Memberikan margin sebesar 1rem (4 * 0.25rem).
<code>bg-blue-500</code>	Mengatur background color menjadi biru dengan tingkat kecerahan 500 (standar dari skala warna Tailwind).
<code>text-white</code>	Mengubah warna teks menjadi putih .
<code>px-4</code>	Memberikan padding horizontal (<i>padding-left & padding-right</i>) sebesar <code>1rem</code>
<code>py-2</code>	Memberikan padding vertical (<i>padding-top & padding-bottom</i>) sebesar <code>0.5rem</code>
<code>rounded</code>	Memberikan border-radius default untuk membuat tombol agak membulat.

▼ Extension Tailwind CSS



- **Auto-suggestion** untuk class Tailwind
- **Hover preview** untuk melihat warna, ukuran, dll.
- **Linting & error checking** untuk class yang salah



▼ Kategori Tailwind CSS

Tailwind CSS menawarkan fleksibilitas dalam kustomisasi desain. Agar lebih mudah dipahami dan digunakan secara efektif, kita akan membaginya ke dalam beberapa kategori sebagai berikut:

▼ 1 Spacing (Margin & Padding)

- ♦ Mengatur **jarak luar (margin)** dan **jarak dalam (padding)** elemen.
- `m-4` → Margin `1rem`
- `p-2` → Padding `0.5rem`
- `mx-4` → Margin horizontal `1rem` (left & right)
- `py-3` → Padding vertical `0.75rem` (top & bottom)

Contoh:

```
function Spacing(){
  return (
    <div className="bg-white shadow-lg p-6 m-4 rounded-lg">
      <h2 className="text-lg font-semibold">Card Title</h2>
      <p className="mt-2 text-gray-600">Ini adalah contoh penggunaan padding dan margin di Tailwind.</p>
    </div>
  )
}
```

▼ 2 Typography (Teks & Font)

- ♦ Mengatur ukuran, ketebalan, warna, dan gaya teks.
- `text-lg` → Ukuran teks besar
- `font-bold` → Teks tebal

- `text-gray-500` → Warna abu-abu
- `uppercase` → Huruf kapital semua

Contoh:

```
function Typography(){
  return (
    <div>
      <h1 className="text-3xl font-bold text-blue-600">Tailwind Typography</h1>
      <p className="text-gray-600 text-lg mt-2">Be
lajar Tailwind sangat menyenangkan dan cepat!</p>
    </div>
  )
}
```

▼ 3 Border & Radius (Bentuk Elemen)

- ♦ Menambahkan garis tepi dan membuat sudut membulat.
- `border` → Garis tepi default
- `border-2` → Tebal border 2px
- `border-red-500` → Warna border merah
- `rounded-lg` → Membuat sudut membulat

Contoh:

```
function BorderRadius(){
  return (
    <button className="border-2 border-blue-500 text-blue-500 px-4 py-2 rounded-lg"> Klik Saya </button>
  )
}
```

▼ 4 Background & Colors

- ♦ Mengatur warna latar belakang dan teks.
- `bg-blue-500` → Latar belakang biru
- `bg-gradient-to-r from-green-400 to-blue-500` → Gradien hijau ke biru
- `text-white` → Teks putih

Contoh:

```
function BackgroundColors(){
  return(
    <div className="bg-blue-500 text-white p-6 rounded-lg shadow-lg">
      <h3 className="text-xl font-bold">Tailwind Colors</h3>
      <p className="mt-2">Belajar Tailwind itu seru dan fleksibel!</p>
    </div>
  )
}
```

▼ 5 Flexbox & Grid (Layouting)

- ♦ Membantu menyusun tata letak dengan mudah.
- `flex` → Mengaktifkan Flexbox
- `justify-between` → Menyebarakan elemen ke ujung
- `items-center` → Pusatkan elemen secara vertikal
- `grid grid-cols-3` → Membuat layout 3 kolom

Contoh:

```
function FlexboxGrid(){
  return (
    <nav className="flex justify-between bg-gray-800 p-4 text-white">
      <h1 className="text-lg font-bold">MyWebsite
    </h1>
      <ul className="flex space-x-4">
        <li><a href="#">Home</a></li>
        <li><a href="#">About</a></li>
        <li><a href="#">Contact</a></li>
      </ul>
    </nav>
  )
}
```

▼ 6 Shadow & Effects

- ♦ Menambahkan bayangan dan efek visual lain.
- `shadow-md` → Bayangan sedang
- `shadow-lg` → Bayangan besar
- `hover:shadow-xl` → Efek bayangan saat hover
- `opacity-50` → Transparansi 50%

Contoh:

```
function ShadowEffects(){
  return (
    <div className="bg-white shadow-lg p-6 rounded-l
g hover:shadow-2xl transition">
      <h3 className="text-xl font-semibold">Hover
me!</h3>
      <p className="text-gray-600 mt-2">Lihat efek
bayangan saat hover.</p>
    </div>
  )
}
```

```
        </div>
    )
}
```

Untuk mendapatkan penjelasan yang lebih lengkap, silahkan akses dokumentasi Tailwind CSS:

<https://tailwindcss.com/docs/>

▼ Reusable Component

Reusable Component merupakan suatu komponen yang bisa digunakan berulang kali, sehingga pengembangan bisa lebih efisien dan terstruktur.



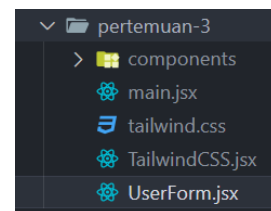
Umumnya Reusable Component menerapkan konsep *Single Responsibility* dimana setiap komponen bertanggung jawab untuk suatu tugas khusus agar komponen dapat digunakan kembali pada tempat yang dimana memiliki tugas dan fungsi yang hampir sama.

Silahkan buat sebuah file baru dengan nama

`UserForm.jsx` lalu masukkan kode berikut

▼ `UserForm.jsx`

```
export default function UserForm
() {
  return (
    <div className="flex flex-col
items-center justify-center m-5 p
-5 bg-gray-100">
      <div className="bg-white p-
6 rounded-lg shadow-lg w-96">
        <h2 className="text-2xl f
ont-semibold text-center mb-4">Ta
```



```

mbah User</h2>
    <label className="block t
ext-gray-700 font-medium">Nama</l
abel>
    <input
        type="nama"
        placeholder="Masukkan N
ama"
        className="w-full p-2 b
order border-gray-300 rounded mb-
2"
    />
    <label className="block t
ext-gray-700 font-medium">Email</
label>
    <input
        type="email"
        placeholder="Masukkan E
mail"
        className="w-full p-2 b
order border-gray-300 rounded mb-
2"
    />
    <label className="block t
ext-gray-700 font-medium">Tanggal
Lahir</label>
    <input
        type="date"
        placeholder="Masukkan T
anggal Lahir"
        className="w-full p-2 b
order border-gray-300 rounded mb-
4"
    />
    <button className="w-full
bg-green-500 text-white p-2 round

```

```

ed">
        Simpan
      </button>
    </div>
  </div>
);
}

```

Perhatikan penulisan kode pada form berikut:

```

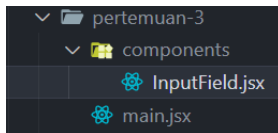
1  export default function UserForm() {
2    return (
3      <div className="flex flex-col items-center justify-center m-5 p-5 bg-gray-100">
4        <div className="bg-white p-6 rounded-lg shadow-lg w-96">
5          <h2 className="text-2xl font-semibold text-center mb-4">Tambah User</h2>
6          <label className="block text-gray-700 font-medium">Nama</label>
7          <input
8            type="nama"
9            placeholder="Masukkan Nama"
10           className="w-full p-2 border border-gray-300 rounded mb-2"
11         />
12         <label className="block text-gray-700 font-medium">Email</label>
13         <input
14           type="email"
15           placeholder="Masukkan Email"
16           className="w-full p-2 border border-gray-300 rounded mb-2"
17         />
18         <label className="block text-gray-700 font-medium">Tanggal Lahir</label>
19         <input
20           type="date"
21           placeholder="Masukkan Tanggal Lahir"
22           className="w-full p-2 border border-gray-300 rounded mb-4"
23         />
24         <button className="w-full bg-green-500 text-white p-2 rounded">
25           Simpan
26         </button>
27       </div>
28     </div>
29   );
30 }

```

Terlihat bahwa ada pembuatan kode yang berulang pada bagian `<label>` dan `<input>`. Keduanya hanya berbeda di *type* dan *text* saja antara Nama, Email

dan Tanggal Lahir.

Kita akan coba buat komponen yang reusable, sehingga kode bisa lebih efektif. Buat folder baru `components`, lalu buat file `InputField.jsx` dengan isi sebagai berikut:



```
1 export default function InputField( { label, type, placeholder } ) {  
2   return (  
3     <div className="mb-3">  
4       <label className="block text-gray-700 font-medium mb-1">{label}</label>  
5       <input  
6         type={type}  
7         placeholder={placeholder}  
8         className="w-full p-2 border border-gray-300 rounded"  
9       />  
10    </div>  
11  );  
12 }
```



Komponen `InputField` akan menerima 3 props saat di panggil yaitu *label*, *type* dan *placeholder*. Props ini dibutuhkan agar komponen tersebut dapat digunakan di dalam kondisi yang beda-beda.

Terapkan komponen tersebut untuk inputan yang tersedia. `UserForm()` akan terlihat lebih sederhana dan mudah untuk di-maintenance:

```
export default function UserForm() {  
  return (  
    ...  
    <InputField label="Nama" type="text" placeholder  
    ="Silahkan ketik Nama..." />  
  
    <InputField label="Email" type="email" placeholder  
    ="Silahkan ketik EMail..." />  
  
    <InputField label="Tanggal Lahir" type="date" />  
    ...  
  )  
}
```

```
);  
}
```

▼ React Form dengan State

Selanjutnya kita akan menangani input dari user melalui sebuah form lalu menyimpan inputan tersebut ke State melalui Event Handling dan menampilkan UI sesuai dengan value State.



- **State** merupakan tempat menyimpan dan mengelola data yang bisa berubah dalam suatu komponen React.
- Jika terdapat 2 atau 3 inputan yang perlu di ambil value-nya, maka state juga disiapkan sebanyak inputan tersebut
- Jika form diisi oleh user, kita perlu **menyimpan data tersebut ke state**, agar dapat diolah seperti validasi, perhitungan atau mengirim ke API.

Buat sebuah file baru dengan nama `HitungGajiForm.jsx` lalu isi dengan kode berikut:

▼ `HitungGajiForm.jsx`

```
export default  
function Hitung  
GajiForm() {  
  return (  
    <div cl  
assName="flex f  
lex-col items-c  
enter justify-c  
enter m-5 p-5 b
```

```
g-gray-100">  
    <div  
v className="bg  
-white p-6 rounded-lg shadow-l  
g w-96">
```

```
<h2 className  
="text-2xl font  
-semibold text-  
center mb-4 tex  
t-gray-700">Hit  
ung Gaji Bersih  
</h2>
```

```
<div className  
="mb-4">
```

```
<label classNam  
e="block text-g  
ray-700 font-me  
dium mb-1">
```

Gaji Pokok

```
</label>
```

```
<input
```

```
type="number"
```

```
placeholder="Ma  
sukkan jumlah g  
aji "
```

```
className="w-full p-2 border border-gray-300 rounded focus:ring-2 focus:ring-blue-500"
```

```
/>
```

```
</div>
```

```
<div className="mb-4">
```

```
<label className="block text-gray-700 font-medium mb-1">
```

```
Pajak: <b class="text-red-500">11%</b>
```

```
</label>
```

```
</div>
```

```
<div className="mt-4 p-3 bg-blue-100 border-l-4 border-blue-500 text-blue-700">
```

```
<p className="f
```

```

ont-semibold">

Total Take Home
Pay (THP): Rp 0

</p>

</div>

</div>

iv>

</div>

);

}

```

Kita akan menerapkan State untuk melakukan perhitungan THP berdasarkan ketikkan User dan nominal pajak yang ditetapkan. Sesuaikan kode diatas dengan menerapkan perubahan berikut:

1. Deklarasikan State `gaji` dan pengolahan perhitungan:

```

1  import { useState } from "react";
2
3  export default function HitungGajiForm() {
4
5      const [gaji, setGaji] = useState("");
6
7      const pajak = 0.11
8      const totalGaji = gaji-(gaji*pajak)
9
10     return (
11         ...
12     );
13 }
14

```

- `gaji` merupakan variabel yang menyimpan nilai inputan pengguna.
- `setGaji` adalah fungsi yang dipanggil saat event `onChange` digunakan untuk memperbarui value dari variabel `gaji`.

- `useState("")` berarti nilai awal dari variabel `gaji` adalah **string kosong** (`""`).
- `pajak` dan `totalGaji` merupakan rule dalam pengolahan data `gaji`

2. Sesuaikan inputan Gaji dengan menerapkan Event Handling `onChange` :

```
<input
  type="number"
  placeholder="Masukkan jumlah gaji"
  className="w-full p-2 border border-gray-300 rounded focus:ring-2 focus:ring-blue-500"
  onChange={(e) => setGaji(e.target.value)}
/>
```

- Jenis Event Handling-nya berupa `onChange` pada inputan gaji,
- Setiap user melakukan input, function `setGaji()` akan dipanggil dengan menjadikan value ketikkan sebagai parameter-nya
- `e.target.value` ini digunakan mendapatkan value ketikkan user
- State `gaji` akan diperbaharui secara otomatis setiap kali user mengetik di inputan tersebut

3. Sesuaikan bagian THP dengan mengambil dari variabel `totalGaji`

```
<div className="mt-4 p-3 bg-blue-100 border-1-4 border-blue-500 text-blue-700">
  <p className="font-semibold">
    Total Take Home Pay (THP): Rp {totalGaji.toLocaleString()}
  </p>
</div>
```

- Pada bagian ini `totalGaji` ditampilkan dengan menerapkan `toLocaleString()` agar angka yang tampil lengkap dengan pemisah ribuan

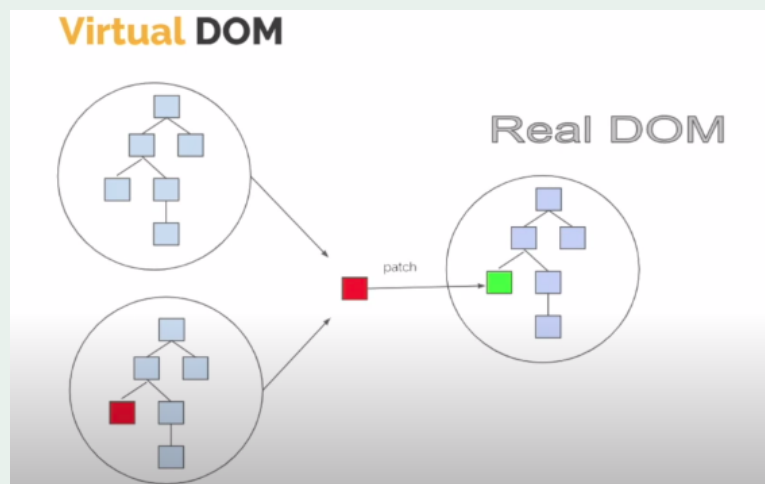
4. Proses Re-render ulang UI akan terjadi setiap perubahan State

Ketika State `gaji` berubah saat adanya ketikkan oleh User, React akan me-re-render ulang komponen

`HitungGajiForm()` sehingga perhitungan `totalGaji` yang juga ada di function tersebut akan dihitung kembali dan di-update ke UI yang menampilkannya.



Perlu diperhatikan bahwa setiap re-render terjadi, React hanya memperbarui bagian yang berubah (*dalam hal ini hanya bagian nominal THP*), karena React melakukan perbandingan sebelum dan sesudah perubahan di Virtual DOM sehingga perubahan terasa cepat dan responsif



▼ Conditional Rendering

Sebelumnya kita sudah mengetahui bahwa pada komponen `HitungGajiForm.jsx` memiliki sebuah state `gaji` yang menyimpan nominal dari gaji yang diinputkan.

Sekarang kita akan coba pasang kondisional sebagai berikut:

- Jika `gaji` belum diinputkan atau bernilai 0, maka tampilkan informasi "Silakan masukkan gaji yang valid (tidak boleh kosong atau negatif)"

- Jika `gaji` berisi nilai yang valid, maka tampilkan informasi hasil perhitungan THP

Lakukan penyesuaian pada `div` bagian "Total Take Home Pay (THP)" menjadi seperti berikut:

```
{!gaji ? (  
  <div className="mt-4 p-3 bg-red-100 border-1-4 border-red-500 text-red-700">  
    <p className="font-semibold">  
      Silakan masukkan gaji yang valid (tidak boleh kosong atau negatif).  
    </p>  
  </div>  
) : (  
  <div className="mt-4 p-3 bg-blue-100 border-1-4 border-blue-500 text-blue-700">  
    <p className="font-semibold">  
      Total Take Home Pay (THP): Rp {totalGaji.toLocaleString()}  
    </p>  
  </div>  
)}
```

Saat inputan kosong

Hitung Gaji Bersih

Gaji Pokok

Masukkan jumlah gaji

Pajak: 11%

Silakan masukkan gaji yang valid (tidak boleh kosong atau negatif).

Saat inputan sesuai dengan kebutuhan

Hitung Gaji Bersih

Gaji Pokok

Pajak: 11%

Total Take Home Pay (THP): Rp 89,000



Conditional Rendering tetap mengharuskan komponen yang dikembalikan terdiri dari satu elemen pembungkus, seperti pada functional component pada umumnya.
